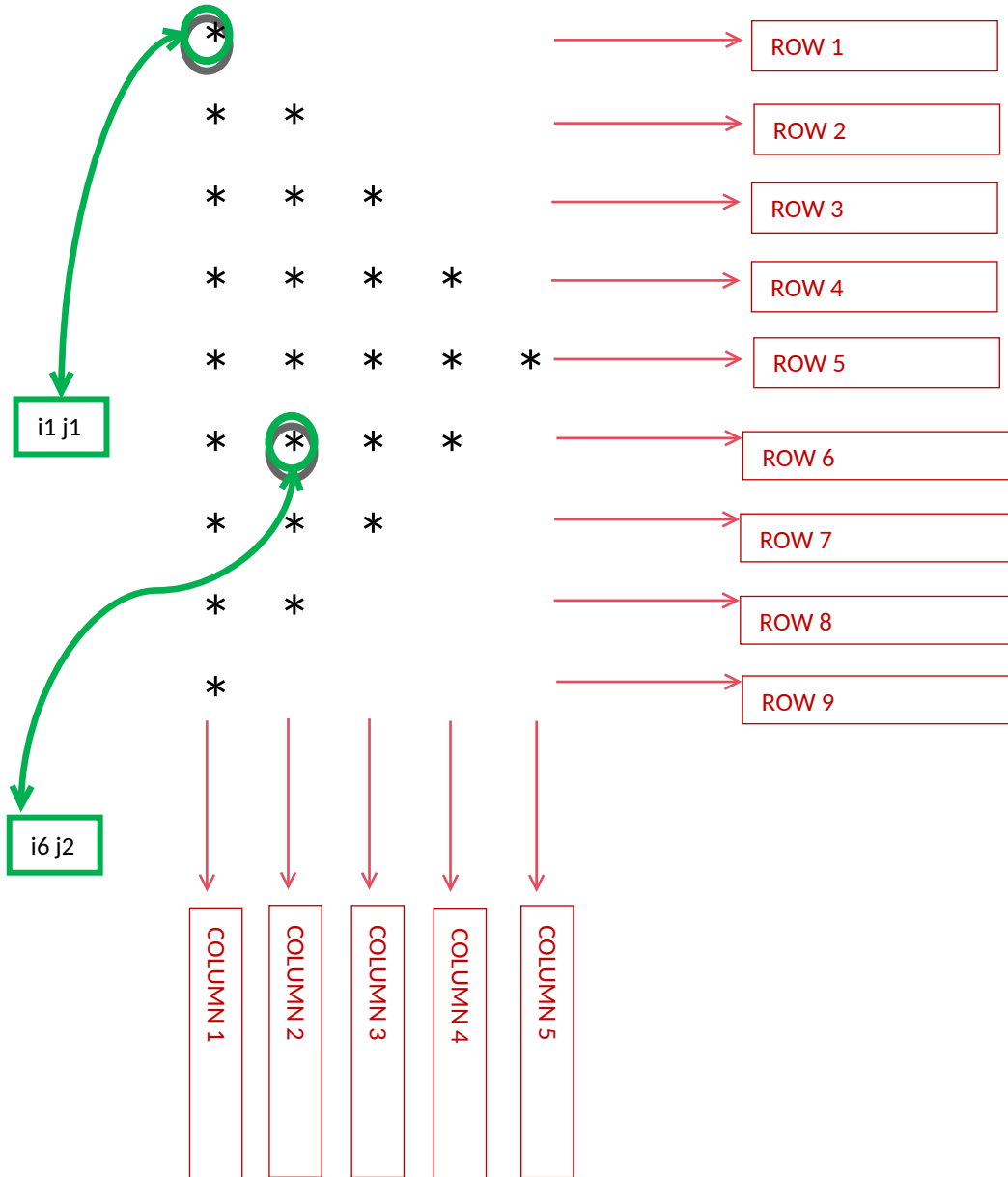


WEEK 01 MODULE
Programming Practice

TOPIC : 1 'Star Pattern'

1. HALF DIAMOND



- Half Diamond = Right Angled Triangle + Inverted Triangle

OR

- Half Diamond = Half Diamond Upper + Half Diamond Lower

1. Problem Analysis :-

i. Pattern Structure:

- The output forms a half diamond
- Unlike the Diamond Pattern, this pattern is not centered and does not require leading spaces.

Instead, it consists of two separate parts:

Upper Half Diamond + Lower Half Diamond

OR

Upper Half Right Angled Triangle + Lower half Inverted Triangle

- The upper half expands downwards
- The lower half contracts downwards

ii. Execution Flow:

Step 1: Observe the Grid Concept:

- For n = 5

```
*  
  
* *  
  
* * *  
  
* * * *  
  
* * * * *  
  
* * * *  
  
* * *  
  
* *  
  
*
```

Upper Half Diamond :-

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Lower Half Diamond :-

```
* * * *
* * *
* *
*
```

- Each cell in the grid represents a unique coordinate (i, j), where i denotes the row index and j denotes the column index.

-The pattern consist of only one visible region

✦ Region 1: Stars

These form the visible half diamond

For n = 5

Row 0 : [*]

Row 1 : [* *]

Row 2 : [* * *]

Row 3 : [* * * *]

Row 4 : [* * * * *]

Row 3 : [* * * *]

Row 2 : [* * *]

Row 1 : [* *]

Row 0 : [*]

UPPER HALF :-

- Stars increase by 1 in every row.

LOWER HALF :-

- Stars decrease by 1 in every row.

-> Row Analysis :-

Upper Half :-

Row Index (I)	0	1	2	3	4
Stars	1	2	3	4	5

RELATIONSHIP :-

- Stars = i + 1

Lower Half :-

Row Index (I)	3	2	1	0
Stars	4	3	2	1

RELATIONSHIP :-

- Stars = i + 1

The formulas remain exactly the same for both halves.

Only the direction of row traversal changes.

Upper Half:

$$i = 0 \rightarrow n - 1$$

Lower Half:

$$i = n - 2 \rightarrow 0$$

Step 2: Traverse the grid row by row.

- The outer loop is responsible for generating rows.

Generate the upper half using the Upper Half Diamond OR Upper Half Right Angled Triangle.

- Traverse rows from 0 to n-1.
- Print the required stars.

After the upper diamond is completed, generate the lower half.

- Traverse rows from n-2 down to 0.
- Print the required stars.

Step 3: After all stars of the current row have been printed, move the cursor to the next line.

- This ensures that the next row starts on a new line.

Step 4: Repeat the process for all rows until the outer loop terminates.

2. Condition Analysis

No conditional statements are required.

There is no need for if(...)

- Every position visited by the star loop prints a star.
- The pattern is controlled entirely through loop limits.

3. Core Logic Transformation

Right Angled Triangle Pattern:-

```
for(int j = 0; j < i + 1; j++)
```

- Meaning:

Stars increase by 1 in every row.

Inverted Triangle Pattern:-

```
for(int j = 0; j < n - i; j++)
```

- Meaning:

Stars decrease by 1 in every row.

Half Diamond Pattern:-

```
for(int i = 0; i < n; i++)
```

- Meaning:

Generates the expanding upper triangle.

```
for(int i = n - 2; i >= 0; i--)
```

- Meaning:

Generates the shrinking lower triangle.

The formula for printing stars remains:

```
for(int j = 0; j < i + 1; j++)
```

Only the row traversal direction changes.

This creates the complete half diamond shape

4. Program :-

```
package week1_module;
import java.util.Scanner;
    public class StarDiamond {

        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter the number of rows: ");
            int n = sc.nextInt();
            // Upper Pyramid
            for(int i = 0; i < n; i++) {

                for(int j = 0; j < n - i - 1; j++) {
                    System.out.print(" ");
                }

                for(int j = 0; j < 2 * i + 1; j++) {
                    System.out.print("* ");
                }

                System.out.println();
            }

            // Lower Inverted Pyramid
            for(int i = n - 2; i >= 0; i--) {

                for(int j = 0; j < n - i - 1; j++) {
                    System.out.print(" ");
                }

                for(int j = 0; j < 2 * i + 1; j++) {
                    System.out.print("* ");
                }

                System.out.println();
            }

            sc.close();
        }
    }
```

✦ Why $n - 2$?

Many students write:

```
for(int i = n - 1; i >= 0; i--)
```

and get:

```
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * * *
* * * *
* * *
* *
*
*
```

here the Middle row is printed twice
as Upper half already printed:

```
* * * * *
```

So we start from: $i = n - 2$

which skips the duplicate middle row.

hence obtained figure:-

```
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * *
* * *
* *
*
```

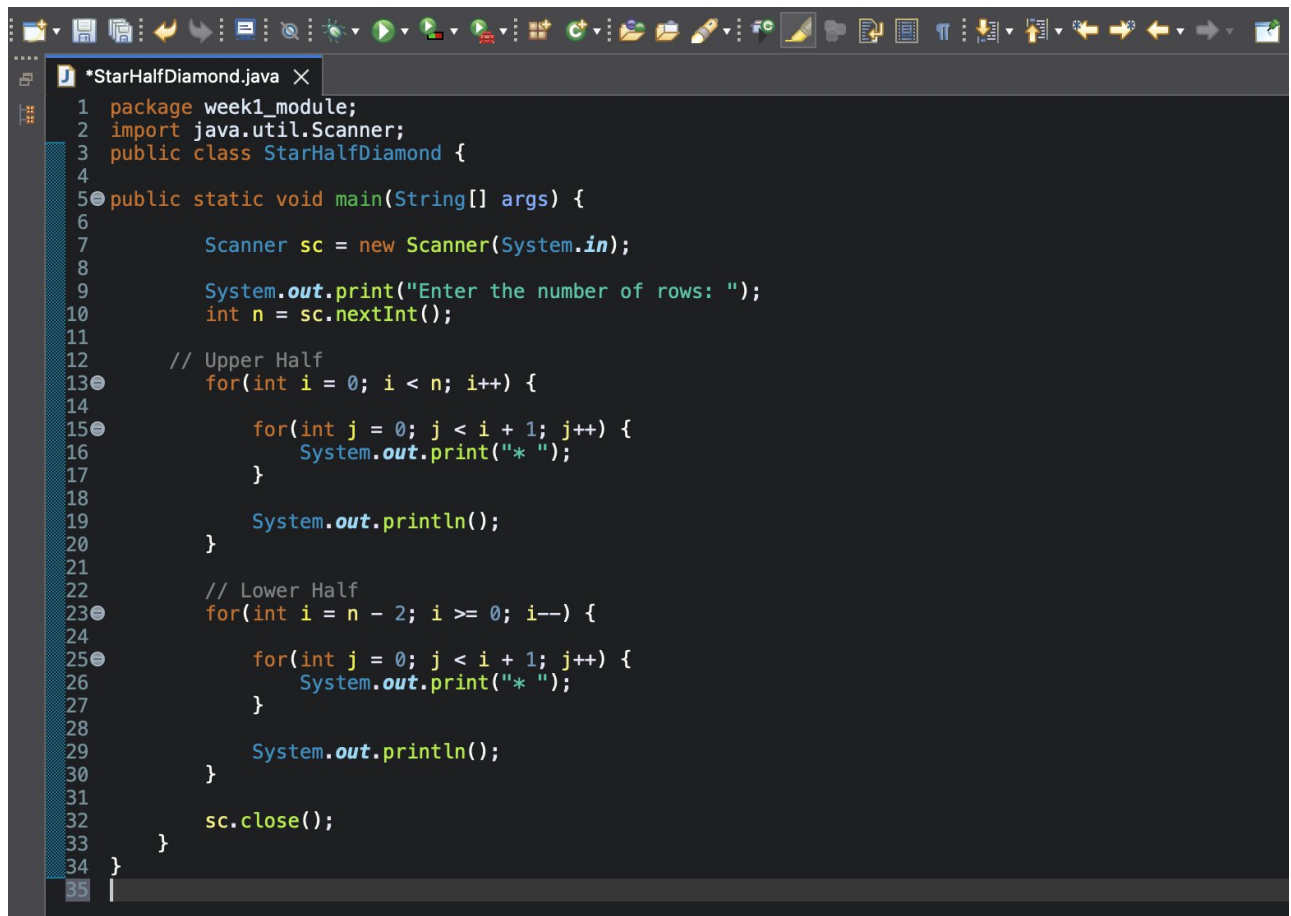
5. Program Output :-

Enter the number of rows: 5

```
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * *
* * *
* *
*
```

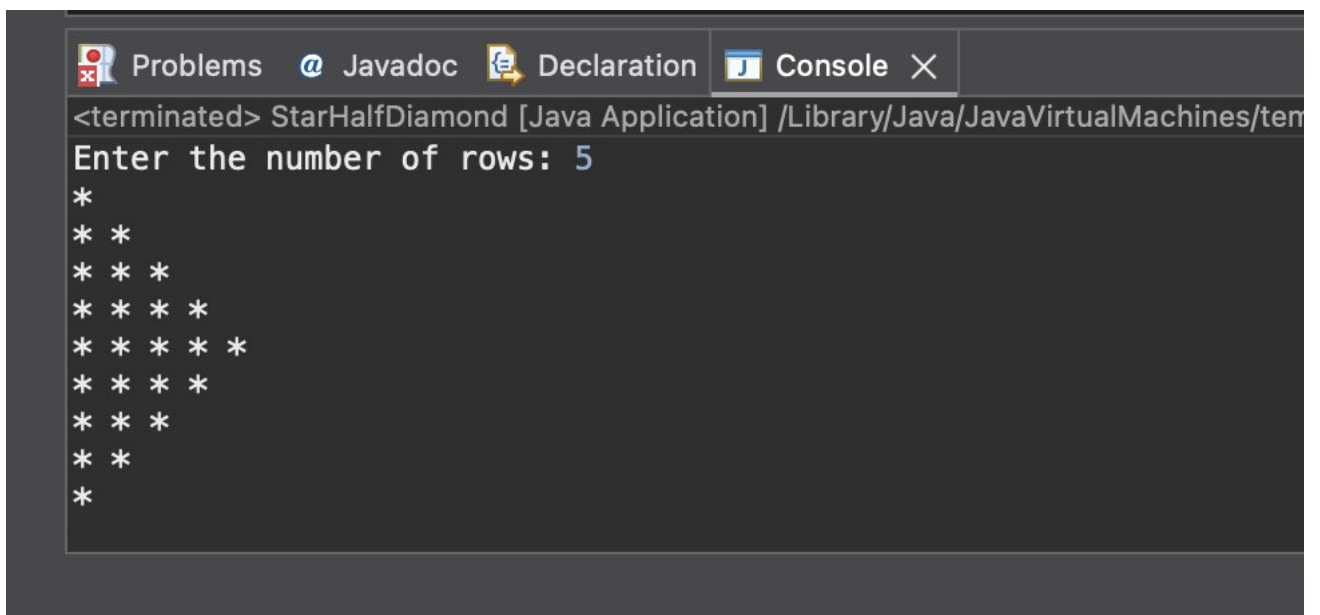
6. ScreenShot :-

i. Program :-



```
1 package week1_module;
2 import java.util.Scanner;
3 public class StarHalfDiamond {
4
5     public static void main(String[] args) {
6
7         Scanner sc = new Scanner(System.in);
8
9         System.out.print("Enter the number of rows: ");
10        int n = sc.nextInt();
11
12        // Upper Half
13        for(int i = 0; i < n; i++) {
14
15            for(int j = 0; j < i + 1; j++) {
16                System.out.print("* ");
17            }
18
19            System.out.println();
20        }
21
22        // Lower Half
23        for(int i = n - 2; i >= 0; i--) {
24
25            for(int j = 0; j < i + 1; j++) {
26                System.out.print("* ");
27            }
28
29            System.out.println();
30        }
31
32        sc.close();
33    }
34 }
35
```

ii. Program's Output :-



```
<terminated> StarHalfDiamond [Java Application] /Library/Java/JavaVirtualMachines/tem
Enter the number of rows: 5
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```